

Angular 16 Project Assignment: Student Management System

This project is designed to help you practice the core concepts you learned in Angular. You must build a small Student Management System that demonstrates component communication, routing, services, observables, and Angular template features.

Project Goal

Build an Angular application that allows users to manage students. The system should allow displaying, adding, deleting, and viewing details of students.

Required Pages (Routing)

- Home Page (/home) – Simple welcome page.
- Students Page (/students) – Displays a table of all students.
- Add Student Page (/add-student) – Form to add a new student.
- Student Details Page (/student/:id) – Displays full information about a student.

Required Components

- NavbarComponent – Navigation between pages.
- StudentsComponent – Main page that contains the student table.
- StudentTableComponent – Responsible for displaying the table of students.
- AddStudentComponent – Contains the form used to add a student.
- StudentDetailsComponent – Displays detailed information about one student.

Student Data Structure

- id
- firstName
- lastName
- age
- gender
- email
- phone
- address
- city
- country
- department

- level
- GPA
- enrollmentDate
- isActive

Example Student Object

```
{
  id: 1,
  firstName: "Ali",
  lastName: "Hassan",
  age: 21,
  gender: "Male",
  email: "ali.hassan@email.com",
  phone: "01000000000",
  address: "12 Main Street",
  city: "Cairo",
  country: "Egypt",
  department: "Computer Science",
  level: 3,
  GPA: 3.2,
  enrollmentDate: "2023-09-01",
  isActive: true
}
```

Students Service

You must create a StudentsService that stores the students data. The service must use BehaviorSubject and Observable to manage and share the data between multiple components.

Student Table Requirements

- Display students in a table format.
- Use *ngFor to render table rows.
- Show columns such as: ID, Name, Age, Department, GPA, Actions.
- Actions column should contain buttons such as View and Delete.

Add Student Page

- Create a form that allows adding a new student.
- Fields should include: name, age, department, email, phone, GPA.
- When the form is submitted, the new student should be added to the service.
- The student table must update automatically.

Student Details Page

When the user clicks 'View' in the table, the application should navigate to /student/:id and display full student information using the route parameter.

Component Communication Requirements

- Use @Input to send data from a parent component to a child component.
- Use @Output with EventEmitter to send events from a child component to a parent component.

Angular Features You Must Use

- Interpolation
- Property Binding
- Event Binding
- Structural Directives (*ngIf, *ngFor)
- Angular Routing
- Services
- BehaviorSubject
- Observable
- Async Pipe
- @Input
- @Output

Bonus Features (Optional)

- Search students by name.
- Filter students by department.
- Edit student information.
- Improve the UI design.

Grading Criteria

- Routing implementation – 10 points
- Component structure – 10 points
- Table implementation using ngFor – 10 points
- Add student feature – 10 points
- Service with BehaviorSubject – 15 points
- Async pipe usage – 10 points
- Input/Output communication – 10 points
- Delete student feature – 10 points
- UI design – 5 points
- Code quality and organization – 10 points